ARQUITETURA DE GOVERNANÇA • SSOT

Anatomia da Pasta `docs/`

A Arquitetura de Conhecimento, Contratos e Rastreabilidade do LegoLoop

---

Dissecação das 12 Subpastas do Diretório de Documentação:

Por que cada pasta existe? • Para que cada pasta foi criada?

Quais tipos de artefatos residem dentro de cada uma? • Como a esteira as consome?

Repositório: LegoLoop Engine

Data: 2026-11-05

Padrão de Nomenclatura: ISO 8601 (`YYYY-MM-DD_...`)

Governança: AGENTS.md (Regras 6, 8 e 14)

Raiz Auditada:

`C:\Users\trcnologia\Desktop\LegoLoop\docs`

Status: Manual Canônico de Referência

Sumário do Manual

1. A Filosofia da Pasta `docs/` no LegoLoop 3

1.1 Por Que a Pasta `docs/` Existe no Ecossistema? 3

1.2 As Leis de Governança Aplicadas a `docs/` 3

2. Mapa Geral da Topologia da Pasta `docs/` 5

2.1 O Documento na Raiz de `docs/`: O Padrão-Ouro da Stack 5

3. As Quatro Pastas da Esteira Canônica (A Cascata) 6

3.1 `docs/intents/` — A Pasta das Declarações de Intenção 6

3.2 `docs/specs/` — A Pasta das Especificações Executáveis BDD 6

3.3 `docs/plans/` — A Pasta dos Planos de Fatiamento Vertical 8

3.4 `docs/routes/` — A Pasta das Matrizes Isomórficas Executáveis 8

4. As Pastas de Governança, Qualidade e Evidência 10

4.1 `docs/templates/` — O Centro de Modelos Canônicos (SSOT) 10

4.2 `docs/reports/` — A Pasta de Relatórios e Evidências Forenses 10

4.3 `docs/sessions/` — A Pasta de Telemetria e Histórico de Sessões 12

4.4 `docs/manuals/` — A Pasta dos Manuais Operacionais e Loops 12

5. As Pastas de Suporte, Referência e Pesquisa 14

5.1 `docs/references/` — A Pasta de Achados e Engenharia Reversa 14

5.2 `docs/prompts/` — A Pasta dos Prompts Mestres de Engenharia 14

5.3 `docs/examples/` — A Pasta de Projetos de Referência Concretos 15

5.4 `docs/archive/` — O Cemitério Histórico de Versões Descontinuadas 15

6. Matriz Geral de Rastreabilidade e Artefatos da Pasta `docs/` 17

7. O Fluxo de Dados e Interação Entre as Pastas 19

8. Conclusão: A Documentação Como Código Executável 19

1. A Filosofia da Pasta `docs/` no LegoLoop

1.1 Por Que a Pasta `docs/` Existe no Ecossistema?

Em projetos de software comuns, a documentação é tratada como um subproduto tardio, incompleto ou descartável ("leia o código"). Em sistemas autônomos baseados em agentes e Modelos de Linguagem (LLMs), essa negligência é fatal. Sem uma topologia documental rigorosa:

1. **Amnésia e Deriva Arquitetural:** Agentes perdem o contexto de decisões tomadas horas antes e refatoram componentes em direções conflitantes.
2. **Poluição do Código-Fonte:** Comentários longos, prompts e especificações misturam-se com arquivos de produção Go/TypeScript, inflando árvores de dependência e confundindo analisadores estáticos.
3. **Falta de Rastreabilidade Forense:** Não se consegue responder quem autorizou uma alteração, por que ela foi feita, quais critérios de teste foram combinados e qual foi a evidência mecânica de sua aprovação.

A pasta `docs/` do LegoLoop nasceu para ser o **Cérebro Auditável e o Cartório de Contratos** do projeto. Ela não contém código de produção; ela armazena as premissas, os contratos executáveis, a memória factual e o histórico imutável de todas as decisões tomadas pelo ecossistema.

1.2 As Leis de Governança Aplicadas a `docs/`

De acordo com o documento soberano `AGENTS.md`, a pasta `docs/` é regida por leis estritas:

- **Regra 6 — Padronização ISO Obrigatória:** Todo artefato em `docs/` (com raras exceções estruturais como `templates/` e `references/`) deve obrigatoriamente utilizar o prefixo de data no formato ISO: `YYYY-MM-DD_<TIPO>_<NOME>.<ext>`. Isso garante ordenação cronológica natural no sistema de arquivos, histórico imediato e elimina ambiguidades de versão.
- **Regra 1 — Zero Stubs:** Nenhum documento explicativo, plano ou especificação pode conter seções com `TODO`, `pass` ou preenchimentos provisórios.
- **Regra 2 — Graph-First:** O grafo semântico indexa os contratos presentes em `docs/` para que agentes recuperem trechos precisos sem carregar arquivos inteiros no prompt.

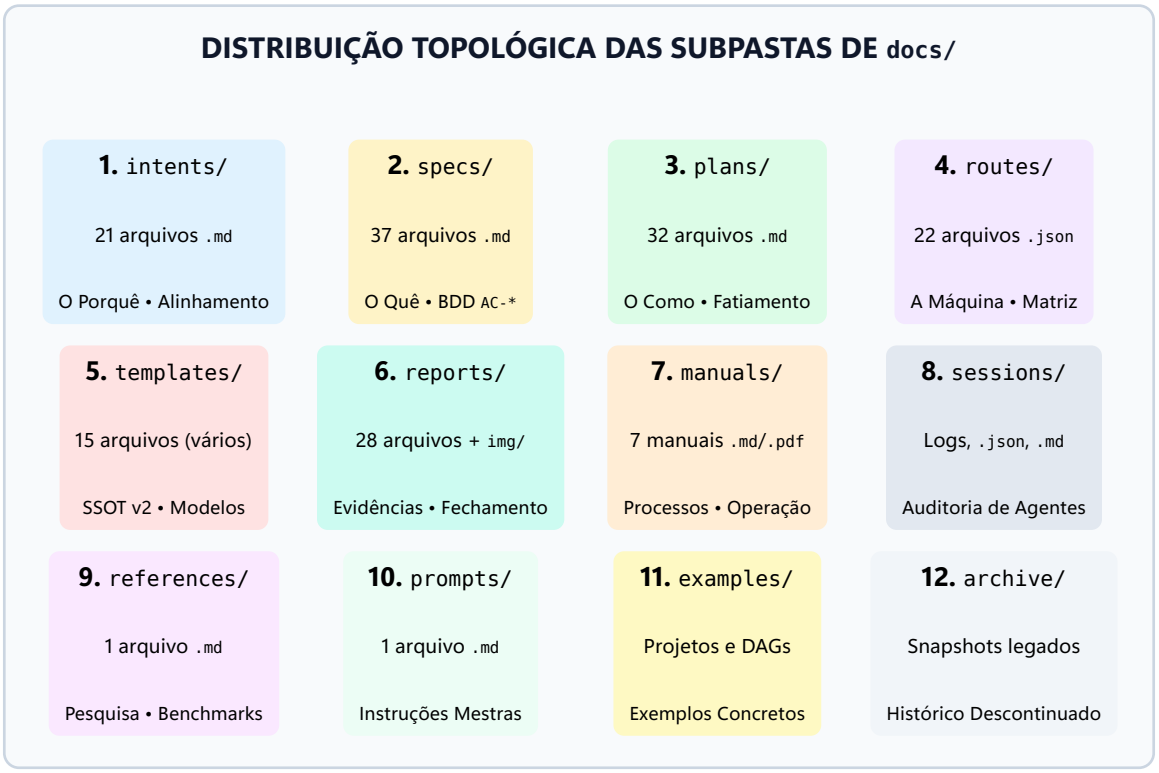
⚖️ O Princípio da Separação em Três Planos

O LegoLoop organiza seu espaço em três planos perfeitamente delimitados:

1. **Plano de Produção (`cmd/`, `internal/`):** Código Go limpo, sem dependências externas, funções pequenas (< 35 linhas).
2. **Plano de Governança e Contratos (`docs/`):** Intenções, especificações BDD, planos, rotas de máquina, relatórios, manuais e sessões.
3. **Plano de Peças Operacionais (`skills/`, `scripts/`, `tools/`):** Habilidades cognitivas e transformações atômicas de automação.

2. Mapa Geral da Topologia da Pasta `docs/`

A pasta `docs/` contém 12 subdiretórios especializados e um documento de diretriz mestra na raiz (`2026-10-17_PADRAO-OURO-STACK-TECNOLOGICA.md`). Cada subpasta desempenha uma função única no ciclo de vida do desenvolvimento.



2.1 O Documento na Raiz de `docs/`: O Padrão-Ouro da Stack

Na raiz do diretório reside o arquivo [`2026-10-17_PADRAO-OURO-STACK-TECNOLOGICA.md`](file:

- **Por que existe:** Nasceu para impedir debates recorrentes e escolhas aleatórias de frameworks a cada projeto gerado no ecossistema (Lei Inviolável 11).
- **O que ele dita:** Define a stack mandatória padrão para qualquer novo software:
  - **Frontend:** TanStack Start / Router + React + TypeScript + Tailwind CSS.
  - **Backend:** Python puro, 100% determinístico.
  - **Database:** SQLite em modo WAL (Write-Ahead Logging).
  - **API & Webhooks:** OpenAPI 3.1 nativo com Swagger Studio e Webhook Studio nativos.
  - **Silêncio do Usuário:** A menos que o plano estruturado exija outra tecnologia, o padrão-ouro é sempre o default inegociável.

3. As Quatro Pastas da Esteira Canônica (A Cascata)

As primeiras quatro pastas formam a coluna dorsal do ciclo de vida de qualquer demanda no LegoLoop, descendo em cascata progressiva: **Intenção** → **Especificação** → **Plano** → **Rota**.

3.1 `docs/intents/` — A Pasta das Declarações de Intenção

🎯 **`docs/#name/`**

Artefatos: 21 arquivos Markdown (.md) em padrão ISO

Por que nasceu e existe? (A Causa & O Problema que Resolve)

Historicamente, demandas de software eram iniciadas por suposições apressadas ou descrições incompletas de uma linha (“melhore o sistema de busca”). Agentes de IA começavam a codificar sem saber qual era o gargalo real, quem sofria com ele e quais eram os limites do escopo. A pasta `intents/` nasceu para ser o **marco zero inegociável**: nenhuma linha de especificação ou código pode existir antes que uma INTENT formal seja gerada, atingindo no mínimo 95% de confiança na entrevista socrática.

Para que serve e como opera? (O Propósito Funcional & Atores)

- **Para que serve:** Registra o alinhamento de intenção socrático (metodologia Addy Osmani adaptada em PT-BR). Responde categoricamente “por que fazer agora” e “o que NÃO fazer” (não-objetivos).
- **Tipos de artefatos:** Documentos Markdown estruturados no formato canônico `YYYY-MM-DD_INTENT_<NOME>.md`. Atualmente contém 21 arquivos (ex: `2026-10-31_INTENT_BLINDAGEM_WORKSPACE.md`, `2026-11-03_INTENT_LEGOLOOP_FABRICAR.md`).
- **Como é alimentada:** Pelo motor Go `interview.MaterializeIntent` ou pela skill `skills/aidd-grill`.
- **Quem utiliza:** O desenvolvedor humano (que expressa a dor e aprova no Stop-Gate 01) e o motor `specgen` (que lê a intenção aprovada para gerar a especificação técnica).

3.2 `docs/specs/` — A Pasta das Especificações Executáveis BDD

📋 **`docs/#name/`**

Artefatos: 37 arquivos Markdown (.md) com critérios BDD AC-*

Por que nasceu e existe? (A Causa & O Problema que Resolve)

Projetos falham quando os critérios de sucesso são subjetivos. Textos como "o login deve ser intuitivo" ou "o pipeline deve ser rápido" não podem ser validados por testes unitários nem por agentes automatizados. A pasta `specs/` nasceu para transformar ideias abstratas em **especificações executáveis binárias**: cada funcionalidade é descrita com premissas técnicas, fronteiras de ação rígidas e critérios de aceite no formato Given-When-Then.

Para que serve e como opera? (O Propósito Funcional & Atores)

- **Para que serve:** É a única fonte de verdade sobre o comportamento do sistema. Define premissas de arquitetura (Osmani), costuras de teste (Pocock) e critérios binários de aceite (`AC-XX.Y` funcionais e `AC-NF-XX` não funcionais). Todo épico deve ter ao menos um critério de caminho de erro/falha.
- **Tipos de artefatos:** Documentos Markdown no padrão `YYYY-MM-DD_SPEC_<NOME>.md`. Conta com 37 especificações ativas (incluindo a `2026-10-30_SPEC_MASTER.md` que governa todo o LegoLoop).
- **Como é alimentada:** Pelo motor Go `specgen.Generate`, que consome a INTENT aprovada e o código existente do alvo.
- **Quem utiliza:** Aprovada pelo humano no Stop-Gate 02; consumida pelo motor `plangen` para criar os tickets; auditada por gates mecânicos (`has_binary_criteria`).

3.3 `docs/plans/` — A Pasta dos Planos de Fatiamento Vertical

🗺️ **`docs/#name/`**

Artefatos: 32 arquivos Markdown (.md) organizados por Épicos e Tickets

Por que nasceu e existe? (A Causa & O Problema que Resolve)

Muitas ferramentas dividem o desenvolvimento em camadas horizontais desconectadas (ex: criar todo o banco de dados sem nenhuma regra de negócio, ou todas as telas sem backend). Quando os módulos se encontram, nada funciona. A pasta `plans/` nasceu para forçar o **fatiamento em fatias verticais (tracer bullets)**: cada ticket entrega um pedaço completo e verificável de ponta a ponta. Além disso, nasceu para ser a interface em linguagem natural legível para supervisão humana da execução de agentes.

Para que serve e como opera? (O Propósito Funcional & Atores)

- **Para que serve:** Apresenta a estratégia de execução do pipeline: níveis de supervisão (`nano`, `micro`, `epic`, `autonomous`), política de rollback, tetos de tokens e divisão em Épicos com tickets `TICK-NN.M`. Cada ticket fecha exatamente um critério `AC-*` da SPEC. É o espelho 1:1 legível do arquivo de rota em JSON.
- **Tipos de artefatos:** Documentos Markdown no formato `YYYY-MM-DD_PLAN_<NOME>.md`. Conta com 32 planos estruturados (ex: `2026-10-30_PLAN_MASTER.md`, `2026-11-03_PLAN_LEGOLOOP_FABRICAR.md`).
- **Como é alimentada:** Pelo motor Go `plangen.Generate`.
- **Quem utiliza:** Lido e assinado pelo usuário no Stop-Gate 03 antes de disparar qualquer execução em código.

3.4 `docs/routes/` — A Pasta das Matrizes Isomórficas Executáveis

⚡ **`docs/#name/`**

Artefatos: 22 arquivos JSON (.json) com nós isomórficos de execução

Por que nasceu e existe? (A Causa & O Problema que Resolve)

Agentes de software não devem tomar decisões operacionais baseados em texto livre em prosa, pois isso abre margem para alucinações, loops infinitos e esquecimento de parâmetros de isolamento. A pasta `routes/`

nasceu para armazenar a **matriz de execução pura de máquina em JSON**: um grafo de nós estritamente tipado, imutável e verificável que o orquestrador despacha para execução estéril.

Para que serve e como opera? (O Propósito Funcional & Atores)

- **Para que serve:** Conter o grafo de execução consumido pelo comando `legoloop run` e pelo motor `orchestrix_hermetic`. Cada nó declara seu modo (`tdd_red_green`), timeouts, cota máxima de tokens (600 tokens), comando argv de validação (sem shell), contingência dupla de modelos (primário e fallback) e expressão de gate booleano.
- **Tipos de artefatos:** Arquivos JSON conformes ao schema `route.v2.json`, nomeados como `YYYY-MM-DD_ROUTE_<NOME>.json`. Possui 22 matrizes de rota geradas.
- **Como é alimentada:** Gerada simultaneamente ao `PLAN.md` pelo motor `plangen`.
- **Quem utiliza:** O executor de pipelines da CLI (`legoloop run`), o LegoLoop Studio e os subprocessos estéreis de agentes.

4. As Pastas de Governança, Qualidade e Evidência

Além das quatro pastas da cascata, o ecossistema mantém pastas vitais para modelos, comprovação de resultados e auditoria histórica de inteligência artificial.

4.1 `docs/templates/` — O Centro de Modelos Canônicos (SSOT)

📐 **`docs/#name/`**

Artefatos: 15 arquivos (.md, .json, .ps1, .go, .typ, .pdf)

Por que nasceu e existe? (A Causa & O Problema que Resolve)

Na V1, modelos e padrões ficavam espalhados ou duplicados dentro de strings no código Go dos motores, levando a descompassos frequentes quando alguém alterava um formato. A pasta `templates/` nasceu para ser o **Single Source of Truth (SSOT v2.0.0)** absoluto do ecossistema: centralizar os 10 modelos oficiais de governança e peças de forma que o código Go leia diretamente desta pasta via `go:embed`.

Para que serve e como opera? (O Propósito Funcional & Atores)

- **Para que serve:** Fornecer os gabaritos oficiais para as 4 etapas da esteira (`INTENT.md`, `SPEC.md`, `PLAN.md`, `ROUTE.json`) e para a sextilha de peças atômicas (`SKILL.md`, `SCRIPT.ps1`, `HOOK.json`, `MCP.json`, `GATE.json`, `LOOP.json`).
- **Tipos de artefatos:** Documentos Markdown, schemas JSON, scripts PowerShell com BOM, código Go (`templates.go`, `templates_test.go`), arquivos Typst (`.typ`) e manuais em PDF (`.pdf`). Total: 15 arquivos.
- **Como é alimentada:** Mantida sob rigorosa governança humana; qualquer mudança exige que os testes `go test ./docs/templates/...` continuem 100% verdes.
- **Quem utiliza:** O compilador Go (via `templates.go`), a CLI `legoloop fabricar`, o comando `legoloop upgrade` e os desenvolvedores.

4.2 `docs/reports/` — A Pasta de Relatórios e Evidências Forenses

📊 **`docs/#name/`**

Artefatos: 28 arquivos (.md, .json, .pdf) + subpasta de imagens img/

Por que nasceu e existe? (A Causa & O Problema que Resolve)

Na engenharia de software tradicional, é comum afirmar que uma tarefa foi concluída sem exibir provas factuais (ex: "está tudo testado"). Em sistemas de IA, essa declaração vazia é inaceitável. A pasta `reports/` nasceu da **Regra de Verificação de Verdade (AGENTS.md, Regra 8)**: ao concluir qualquer ticket, épico ou demanda, o agente deve gerar um relatório detalhando exatamente o que foi alterado, o blast radius, os comandos reais de teste rodados com seus respectivos códigos de saída (`exit code 0`), além de declarar abertamente o que NÃO foi provado.

Para que serve e como opera? (O Propósito Funcional & Atores)

- **Para que serve:** Registrar o fechamento formal de demandas com evidências empíricas de funcionamento. Contém relatórios de fechamento de épicos, relatórios de auditoria de reputação de peças e capturas de tela da interface visual na pasta interna `reports/img/`.
- **Tipos de artefatos:** Arquivos Markdown (`YYYY-MM-DD_REPORT_<NOME>.md`), arquivos de telemetria em JSON (`.json`), imagens de comprovação visual (`.png`) e relatórios consolidados em PDF. Total: 28 arquivos + pasta `img/`.
- **Como é alimentada:** Pelos agentes e engenheiros ao final de cada esteira (`aidd-delivery`, fechamento de épicos).
- **Quem utiliza:** Desenvolvedor humano para auditoria de entrega e clientes externos para homologação.

4.3 `docs/sessions/` — A Pasta de Telemetria e Histórico de Sessões

⏳ **`docs/#name/`**

Artefatos: 2 arquivos raiz (.json, .md) + subpastas json/, markdown/ e LegoLoop/

Por que nasceu e existe? (A Causa & O Problema que Resolve)

Conversas com assistentes e agentes de IA normalmente se perdem na interface volátil do chat. Quando um bug ocorre ou quando é necessário auditar uma decisão tomada por um modelo há duas semanas, o histórico já não existe mais. A pasta `sessions/` nasceu para garantir a **rastreabilidade forense integral e imutável de todas as conversas do ecossistema**.

Para que serve e como opera? (O Propósito Funcional & Atores)

- **Para que serve:** Serializar, indexar e consolidar cada turno de conversa, parâmetros de tokens, timestamps, IDs de sessão, modelo utilizado e saídas literais geradas durante os atendimentos.
- **Estrutura interna:**
  - `historico_geral.json`: Banco consolidado com metadados de todas as sessões.
  - `INDICE_GERAL.md`: Índice legível em Markdown listando cada atendimento, horário e escopo.
  - `json/`: Transcripts literais em JSON Lines (`.jsonl` / `.json`).
  - `markdown/`: Sessões formatadas em leitura humana contínua (`.md`).
  - `LegoLoop/`: Histórico dedicado às sessões do próprio produto.
- **Como é alimentada:** Pelo script global de governança `faz-session --id <ID> --format all` e pela skill `skills/aidd-session`.
- **Quem utiliza:** Desenvolvedor para retrospectivas (`aidd-retro`), rotação de contexto (`aidd-handoff`) e auditoria de custo.

4.4 `docs/manuals/` — A Pasta dos Manuais Operacionais e Loops

📚 **`docs/#name/`**

Artefatos: 7 manuais extensos em Markdown (.md) e PDF (.pdf)

Por que nasceu e existe? (A Causa & O Problema que Resolve)

Loops e pipelines modulares requerem guias operacionais profundos que expliquem suas filosofias de design, regras de composição e comandos de disparo. Manter essa documentação espalhada em wikis ou no README raiz causa sobrecarga. A pasta `manuals/` nasceu para ser a **biblioteca de manuais técnicos exaustivos dos grandes subsistemas e loops do LegoLoop**.

Para que serve e como opera? (O Propósito Funcional & Atores)

- **Para que serve:** Instruir humanos e agentes sobre como operar os subsistemas complexos do ecossistema.
- **Tipos de artefatos:** Documentos Markdown analíticos e manuais compilados em PDF (7 manuais principais):
  - `MANUAL_IMPECCABLE_DESIGN_LOOP.md`: Diretrizes visuais e tokens de design.
  - `MANUAL_MATT_POCOCK_SKILLS_LOOP.md`: Engenharia de costuras de teste e tipagem estrita.
  - `MANUAL_ADDY_OSMANI_LOOP.md`: Auditoria de performance web e orçamentos de desempenho.
  - `MANUAL_MEGA_PIPELINE_LEGOLOOP.md`: O orquestrador central do mega pipeline.
  - `MANUAL_FUNCAO_FABRICAR_LEGOLOOP.md`: O comando unificado de fabricação de novas soluções.
  - `MANUAL_HOOKS_HARNESSES_SO.md`: Guia de integração de hooks em diferentes IDEs e sistemas.
  - `MANUAL_TEMPLATES_LEGOLOOP.md` / `.pdf`: O guia completo dos templates v2.
- **Como é alimentada:** Por desenvolvedores seniores e pela skill `skills/aidd-textbook`.
- **Quem utiliza:** Desenvolvedores para onboarding e agentes para consulta profunda de diretrizes.

5. As Pastas de Suporte, Referência e Pesquisa

5.1 `docs/references/` — A Pasta de Achados e Engenharia Reversa

🔬 **`docs/#name/`**

Artefatos: Documentos de investigação técnica e cadernos de campo (.md)

Por que nasceu e existe? (A Causa & O Problema que Resolve)

Durante a construção de ferramentas complexas, os engenheiros realizam investigações técnicas em APIs de terceiros, proxies reversos e arquiteturas externas. Se esses achados ficarem apenas na cabeça do autor, perdem-se na primeira manutenção. A pasta `references/` nasceu para registrar **cadernos de campo e relatórios de pesquisa técnica aprofundada**.

Para que serve e como opera? (O Propósito Funcional & Atores)

- **Para que serve:** Armazenar documentações de referência externa, benchmarks de latência, análises de pacotes e comportamentos não-documentados de gateways (ex: `achados_9router.md`, que disseca como o gateway 9Router manipula streaming de tokens, cabeçalhos de autenticação e parâmetros de esforço cognitivo).
- **Tipos de artefatos:** Documentos Markdown investigativos (`.md`).
- **Quem utiliza:** Arquitetos de software e agentes de diagnóstico ao resolver falhas de conectividade.

5.2 `docs/prompts/` — A Pasta dos Prompts Mestres de Engenharia

💬 **`docs/#name/`**

Artefatos: Prompts mestres de sistema e diretrizes de execução (.md)

Por que nasceu e existe? (A Causa & O Problema que Resolve)

Prompts longos e instruções de sistema para agentes de execução não devem ser codificados como literais gigantescos dentro do código Go nem passados de forma ad-hoc no terminal. A pasta `prompts/` nasceu para conter as **instruções mestras de sistema (System Prompts) auditáveis e versionadas**.

Para que serve e como opera? (O Propósito Funcional & Atores)

- **Para que serve:** Guardar prompts mestres de execução hermética (ex: `2026-10-30_PROMPT_IMPLEMENTATION.md`), que determinam o comportamento estrito de agentes durante a fase de codificação Red-Green TDD, proibindo expressamente a geração de stubs e impondo o estilo telegráfico (Caveman Thinking).
- **Tipos de artefatos:** Documentos Markdown padronizados com o prefixo ISO.
- **Quem utiliza:** Consumido pelos motores de geração de prompt do LegoLoop ao acionar subloops de codificação.

5.3 `docs/examples/` — A Pasta de Projetos de Referência Concretos

💡 **`docs/#name/`**

Artefatos: Casos reais completos: pipeline.dag.json, skills e saídas geradas

---

Por que nasceu e existe? (A Causa & O Problema que Resolve)

A melhor maneira de validar e demonstrar uma esteira de automação é através de projetos reais completos funcionando de ponta a ponta. A pasta `examples/` nasceu para fornecer **projetos de prova de conceito e fixtures de teste realistas**.

Para que serve e como opera? (O Propósito Funcional & Atores)

- **Para que serve:** Demonstrar o funcionamento prático do LegoLoop em cenários de negócio completos. Por exemplo, a pasta `docs/examples/clinica-veterinaria/` contém um pipeline inteiro (`pipeline.dag.json`), seu `LEIAME.md`, suas skills dedicadas (`interview`, `aidd-spec`, `riscos`) e os artefatos reais gerados na subpasta `saida/` (`BRIEFING.md`, `CHECKLIST.txt`, `RISCOS.md`, `SPEC.md`).
- **Tipos de artefatos:** Grafos DAG em JSON, skills completas em subpastas, artefatos gerados em Markdown e texto.
- **Quem utiliza:** Usada como baseline de testes de integração, documentação viva para novos usuários e testes de regressão de pipelines.

5.4 `docs/archive/` — O Cemitério Histórico de Versões Descontinuadas

📦 **`docs/#name/`**

Artefatos: Snapshots históricos organizados por data de descontinuação

Por que nasceu e existe? (A Causa & O Problema que Resolve)

Em grandes refatorações (como a transição da V1 para a V2 do LegoLoop em novembro de 2026), dezenas de artefatos antigos de planos, intenções e rotas tornam-se obsoletos. Apagá-los do disco impediria a auditoria forense do passado; mantê-los nas pastas ativas causaria confusão para os agentes e para os scanners de assets. A pasta `archive/` nasceu como um **cofre de quarentena histórica de artefatos descontinuados**.

Para que serve e como opera? (O Propósito Funcional & Atores)

- **Para que serve:** Guardar snapshots imutáveis de artefatos anteriores a uma migração estrutural. Por exemplo, a subpasta `docs/archive/2026-11-03/` preserva as antigas subpastas `intents/`, `plans/`, `routes/` e `specs/` exatamente como existiam antes da unificação dos templates V2.
- **Tipos de artefatos:** Snapshots completos de pastas e documentos Markdown/JSON legados.
- **Quem utiliza:** Apenas consultada em auditorias históricas; expressamente ignorada por rotinas de busca ativa e pela esteira de execução corrente.

6. Matriz Geral de Rastreabilidade e Artefatos da Pasta `docs/`

Abaixo, a tabela consolidada correlacionando todas as 12 subpastas da pasta `docs/`:

Subpasta	Por Que Nasceu? (Gênese)	Tipos de Artefato	Para Que Existe? (Propósito)	Momento na Esteira	Atores Principais
`docs/intents/`	Impedir código prematuro sem diagnóstico da dor e sem escopo fechado.	Markdown (`.md`) com padrão ISO	Alinhar a intenção do usuário via entrevista socrática com confiança $\geq 95\%$.	Marco Zero (Fase 1)	Usuário humano e motor `interview.MaterializeIntent`.
`docs/specs/`	Eliminar especificações vagas e não-testáveis que causam alucinações.	Markdown (`.md`) com critérios BDD	Fixar premissas, boundaries de ação e critérios binários inegociáveis (`AC-*`).	Especificação (Fase 2)	Motor `specgen.Generate` e revisor humano (Stop-Gate 02).
`docs/plans/`	Evitar planos horizontais desconexos e permitir auditoria humana da esteira.	Markdown (`.md`) com épicos e tickets	Apresentar fatiamento vertical legível de tickets rastreáveis à SPEC.	Planejamento (Fase 3)	Motor `plangen.Generate` e aprovador humano (Stop-Gate 03).
`docs/routes/`	Substituir prosa em texto livre por matriz rígida executável de máquina.	JSON estruturado (`route.v2.json`)	Grafo de execução tipado com comandos argv puros e contingência dupla.	Execução (Fase 4)	CLI `legoloop run` e executor `orchestrix_hermetic`.
`docs/templates/`	Eliminar templates decorativos da V1 e duplicações de strings no Go.	Markdown, JSON, PS1, Go, Typst, PDF	Centralizar os 10 modelos canônicos oficiais embutidos no binário (`//go:embed`).	Permanente (SSOT)	Compilador Go, `templates_test.go`, comando `upgrade`.
`docs/reports/`	Cumprir a regra de verificação da verdade: provar conclusões com evidências.	Markdown, JSON, PNG (em `img/`), PDF	Registrar entrega de tickets, blast radius e exit codes reais dos testes.	Conclusão de Tickets	Agente de entrega (`aidd-delivery`) e auditor humano.
`docs/manuals/`	Centralizar guias operacionais exaustivos dos grandes loops do sistema.	Markdown analítico (`.md`) e PDF	Instruir humanos e agentes sobre operação e padrões de loops complexos.	Onboarding / Consulta	Desenvolvedores seniores e skill `aidd-textbook`.
`docs/sessions/`	Evitar perda de histórico e permitir auditoria forense total de IAs.	JSON consolidado, JSON Lines, Markdown	Gravar turns literais, telemetria de tokens e transcripts completos de chats.	Tempo real / Pós-sessão	Script `faz-session`, skill `aidd-session`, retro.
`docs/references/`	Registrar cadernos de campo de pesquisas ex-	Markdown investigativo (`.md`)	Armazenar documentações de referência ex-	Pesquisa Arquitetural	Arquitetos de software e agentes de diagnóstico.

	ternas e engenharia reversa.		terna e achados de gateways.		
`docs/prompts/`	Impedir prompts gigantescos espalhados em literais ou passados no terminal.	Markdown com prefixo ISO	Guardar instruções mestras de sistema para agentes em contexto estéril.	Início de Execução	Orquestrador LegoLoop e motores de prompt.
`docs/examples/`	Fornecer projetos de referência completos que provam a esteira na prática.	JSON (DAGs), Skills (`.md`), Saídas	Servir de fixtures para testes e documentação viva de uso do ecossistema.	Testes de Integração	Novos desenvolvedores e suítes de teste E2E.
`docs/archive/`	Preservar o histórico de versões descontinuadas sem poluir o ambiente ativo.	Snapshots de pastas legadas	Cofre de quarentena de artefatos obsoletos pós-refatoração (ex: V1).	Auditoria Histórica	Auditores e historiadores do repositório.

7. O Fluxo de Dados e Interação Entre as Pastas

Para entender a arquitetura em funcionamento, observe como a informação viaja através das pastas de `docs/` durante o desenvolvimento de uma funcionalidade:

DIAGRAMA DE FLUXO DA INFORMAÇÃO DOCUMENTAL

1. Alinhamento de Intenção:

Humano dialoga com o sistema → Consulta `docs/templates/TEMPLATE_INTENT.md` →

Salva o resultado aprovado em `docs/intents/YYYY-MM-DD_INTENT_<NOME>.md`.

2. Elaboração da Especificação Técnica:

O motor `specgen` lê a INTENT e a stack em `docs/2026-10-17_PADRAO-OURO-STACK-TECNOLOGICA.md` →

Usa o gabarito `docs/templates/TEMPLATE_SPEC.md` →

Gera e grava a especificação em `docs/specs/YYYY-MM-DD_SPEC_<NOME>.md`.

3. Geração Dual de Plano e Rota (Espelho Isomórfico):

O motor `plangen` consome a SPEC aprovada →

Gera o espelho humano em `docs/plans/YYYY-MM-DD_PLAN_<NOME>.md` →

Gera a rota mecânica em `docs/routes/YYYY-MM-DD_ROUTE_<NOME>.json`.

4. Execução Hermética e Gravação de Evidências:

O comando `legoloop run` lê a ROUTE → Despacha os nós para execução isolada →

Gera o relatório com exit codes reais em `docs/reports/YYYY-MM-DD_REPORT_<NOME>.md` →

Registra a telemetria do atendimento em `docs/sessions/`.

8. Conclusão: A Documentação Como Código Executável

A arquitetura da pasta `docs/` do LegoLoop representa o estado da arte na engenharia assistida por inteligência artificial. Ela prova que a documentação não é um adereço burocrático; ela é o **sistema imunológico do código-fonte**.

Ao separar rigorosamente o porquê (`intents`), o quê (`specs`), o como (`plans`) e a máquina (`routes`), enquanto mantém os modelos oficiais embutidos no binário (`templates`) e audita toda a telemetria (`sessions` e `reports`), o LegoLoop garante que humanos e modelos de IA trabalhem em harmonia matemática, com zero alucinações, zero stubs e controle irrestrito da qualidade do software.